

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE

Department of Computer Science Engineering

Course Code: 23CS002PC304

Course Title: AI Assisted Coding

Year/Sem: III / II

Assignment Number: 10.1 / 24

Name: O.Narasimha Raju

HT No: 2303A53034

Batch: 46

Lab Objectives

- Analyze and improve code quality using AI
- Identify readability and maintainability issues
- Apply best coding practices
- Refactor code using AI suggestions

Problem 1: Bug Detection

Buggy Code

```
def factorial(n):  
    result = 1  
    for i in range(1, n):  
        result = result * i  
    return result
```

Corrected Code

```
def factorial(n):
    if n < 0:
        return "Invalid"
    result = 1
    for i in range(1, n + 1):
        result *= i
    return result
```

Explanation: Loop missed multiplying n (off-by-one error).

Problem 2: Readability Improvement

Original Code

```
def calc(a, b, c):
    if c == "add":
        return a + b
    elif c == "sub":
        return a - b
```

Improved Code

```
def calculate(a, b, operation):
    if operation == "add":
        return a + b
    elif operation == "sub":
        return a - b
    else:
        return "Invalid"
```

Explanation: Improved naming and clarity.

Problem 3: PEP8 Compliance

Original Code

```
def Checkprime(n):
    for i in range(2, n):
        if n % i == 0:
            return False
    return True
```

Improved Code

```
def check_prime(n):
    if n <= 1:
        return False
    for i in range(2, n):
        if n % i == 0:
            return False
    return True
```

Explanation: Followed naming conventions and added validation.

Problem 4: Code Review

Original Code

```
def processData(d):
    return [x * 2 for x in d if x % 2 == 0]
```

Improved Code

```
def double_even_numbers(numbers):
    if not isinstance(numbers, list):
        return "Invalid input"
    return [x * 2 for x in numbers if isinstance(x, int) and x % 2 == 0]
```

Explanation: Added validation and improved naming.

Problem 5: Performance Optimization

Original Code

```
def sum_of_squares(numbers):
    total = 0
    for num in numbers:
        total += num ** 2
    return total
```

Optimized Code

```
def sum_of_squares(numbers):
    return sum(x * x for x in numbers)
```

Explanation: Using built-in functions improves performance.

Conclusion

AI tools help improve code quality, readability, and performance.

However, developers must verify suggestions to ensure correctness.